

1. Why Type Conversion Matters in Data Engineering

Source data does not always arrive in the format we expect.

For example, an API or CSV may send:

```
customer_id = "101"  
order_amount = "2500.50"  
is_active = "True"
```

These values **look** like an integer, decimal and Boolean, but Python sees all three as strings.

```
print(type(customer_id))  
print(type(order_amount))  
print(type(is_active))
```

Output:

```
<class 'str'>  
<class 'str'>  
<class 'str'>
```

Before calculations, validation or loading into a target schema, these values often need to be converted.

2. What Is Type Conversion?

Type conversion means changing a value from one Python data type to another.

Example:

```
customer_id = "101"  
  
customer_id = int(customer_id)
```

Now:

```
print(type(customer_id))
```

Output:

```
<class 'int'>
```

Important conversion functions:

Function	Converts Value To	Example
<code>int()</code>	Integer	<code>int("101")</code>
<code>float()</code>	Float	<code>float("2500.50")</code>
<code>str()</code>	String	<code>str(501)</code>
<code>bool()</code>	Boolean	Must be used carefully with source text

3. String to Integer

Source systems frequently provide numeric values as strings.

```
source_count = "1500"
```

This cannot safely be used with an integer:

```
target_count = 1475
```

Convert it first:

```
source_count = int(source_count)
```

```
rejected_count = source_count - target_count
```

```
print(rejected_count)
```

Output:

Common Data Engineering Uses

`int()` may be used for:

- Customer IDs
- Batch IDs
- Record counts
- Retry counts
- Page numbers
- File counts
- Partition counts

Example:

```
batch_id = int("501")
retry_count = int("3")
page_number = int("7")
```

Important: Decimal Strings

This works:

```
int("1000")
```

This does not:

```
int("1000.50")
```

Because `"1000.50"` represents a decimal.

Use:

```
float("1000.50")
```

If you later do:

```
int(float("1000.50"))
```

the result becomes:

1000

The decimal portion is removed, so only do this if the business requirement allows it.

4. String to Float

Amounts, percentages, durations and file sizes may arrive as text.

```
order_amount = "2500.50"
```

```
order_amount = float(order_amount)
```

Now:

```
print(type(order_amount))
```

Output:

```
<class 'float'>
```

The value can now participate in calculations:

```
tax_percentage = 18
```

```
tax_amount = order_amount * tax_percentage / 100
```

```
print(tax_amount)
```

Typical Uses of `float()`

- Order amount
- Product price
- Tax amount
- Discount percentage

- File size
 - Processing duration
 - Data-quality score
-

5. Converting Values to Strings

Sometimes numbers need to become text.

Example: creating a dynamic file name.

```
batch_id = 501
```

Desired file name:

```
customer_batch_501.csv
```

One approach:

```
file_name = "customer_batch_" + str(batch_id) + ".csv"
```

A cleaner approach:

```
file_name = f"customer_batch_{batch_id}.csv"
```

`str()` is useful when creating:

- File names
 - Folder paths
 - Log messages
 - Batch identifiers
 - Table names
 - Error messages
-

6. Be Careful with Boolean Conversion

Consider:

```
is_active = "False"
```

A common mistake is:

```
is_active = bool(is_active)
```

You may expect `False`, but Python returns:

True

Why?

Because **any non-empty string is considered True**.

For example:

```
print(bool("True"))
print(bool("False"))
print(bool("No"))
print(bool("0"))
```

All return:

True

This is extremely important when working with source-system data.

7. Safer String-to-Boolean Conversion

Instead of directly applying `bool()`, compare the source text.

```
is_active = "False"
```

```
is_active = is_active.strip().lower() == "true"
```

```
print(is_active)
```

Output:

False

For "True":

```
is_active = "True"
```

```
is_active = is_active.strip().lower() == "true"
```

Output:

True

Handling Multiple Boolean Formats

Different source systems may send:

True
TRUE
Yes
Y
1

False
FALSE
No
N
0

A simple approach for accepted true values:

```
source_value = "YES"
```

```
is_active = (  
    source_value.strip().lower()  
    in ["true", "yes", "y", "1"]  
)
```

Output:

True

The main idea is:

Standardise the source text first, then apply the business rule.

8. Understanding Missing Values

Missing data may arrive in several forms:

None

""

"NULL"

"None"

"NA"

"N/A"

These may mean the same thing from a business perspective, but Python does not treat them as the same value.

None

```
discount = None
```

None represents the absence of a value.

```
print(type(discount))
```

Output:

```
<class 'NoneType'>
```

Empty String

```
discount = ""
```

This is still a string.

It simply contains zero characters.

It may come from:

- Empty CSV fields
 - Empty form fields
 - Missing text fields
 - Poorly formatted source data
-

"NULL" Is Still Text

```
discount = "NULL"
```

Python sees this as a string.

The same applies to:

```
"None"
```

```
"NA"
```

```
"N/A"
```

They may represent missing data logically, but technically they are still strings.

9. Zero and False Are Not Missing Values

This distinction is very important.

```
discount = 0
```

Zero may mean:

The customer received no discount.

That is valid information.

Similarly:

`retry_count = 0`

may mean:

The pipeline has not retried.

Do not automatically treat zero as missing.

Similarly:

`is_active = False`

means the customer may be inactive.

It does **not** automatically mean the information is missing.

Remember

None $\neq$ 0

None $\neq$ False

None $\neq$ ""

Always understand the business meaning before changing a value.

10. Standardising Missing Values

Suppose different records contain:

""

"NULL"

"None"

"N/A"

None

Keeping multiple representations makes validation difficult.

A better approach is to standardise them into one representation:

None

None

None

None

None

Example:

```
discount = "NULL"
```

```
if discount.strip().lower() in ["", "null", "none", "na", "n/a"]:  
    discount = None
```

Now the pipeline has one consistent representation for missing data.

11. Why `strip()` and `lower()` Help

Source data may contain inconsistent formatting.

Example:

```
value = " NULL "
```

Using:

```
value.strip()
```

removes extra spaces.

Result:

```
NULL
```

Using:

```
value.lower()
```

converts it to:

```
null
```

Together:

```
value.strip().lower()
```

make source values easier to compare consistently.

12. Conversion Can Fail

Never assume every source value is valid.

Example:

```
customer_id = "ABC101"
```

```
customer_id = int(customer_id)
```

This fails because "ABC101" cannot be converted into an integer.

Similarly:

```
order_amount = "Two Thousand"
```

```
order_amount = float(order_amount)
```

also fails.

This is why production pipelines need validation and error handling.

13. Simple Validation Before Conversion

For simple positive whole numbers:

```
customer_id = "101"

if customer_id.isdigit():
    customer_id = int(customer_id)
else:
    customer_id = None
```

For:

```
customer_id = "A101"
```

the cleaned value becomes:

None

Limitation of `isdigit()`

```
"-10".isdigit()
"10.5".isdigit()
```

Both return `False`.

So `isdigit()` is useful for simple checks but does not solve every conversion problem.

14. Real Data Engineering Scenario

Suppose an API sends:

```
customer_record = {
    "customer_id": "101",
    "customer_name": " Anurag ",
    "order_amount": "2500.50",
    "is_active": "True",
```

```
"discount": "",  
"last_order_date": None  
}
```

Our target expects:

Field	Expected Type
customer_id	Integer
customer_name	Clean String
order_amount	Float
is_active	Boolean
discount	Float or None
last_order_date	String or None

Clean the Customer ID

```
customer_id = int(customer_record["customer_id"])
```

Result:

101

Type:

int

Clean the Customer Name

Source:

" Anurag "

Clean using:

```
customer_name = customer_record["customer_name"].strip()
```

Result:

"Anurag"

Convert Order Amount

```
order_amount = float(customer_record["order_amount"])
```

Result:

2500.5

Convert Active Status

```
is_active = (  
    customer_record["is_active"]  
    .strip()  
    .lower()  
    == "true"  
)
```

Result:

True

Standardise Discount

```
raw_discount = customer_record["discount"]
```

```
if raw_discount.strip().lower() in ["", "null", "none", "na", "n/a"]:  
    discount = None  
else:
```

```
discount = float(raw_discount)
```

Since the source is an empty string:

```
discount = None
```

15. Raw Record vs Cleaned Record

Raw source:

```
{
  "customer_id": "101",
  "customer_name": " Anurag ",
  "order_amount": "2500.50",
  "is_active": "True",
  "discount": "",
  "last_order_date": None
}
```

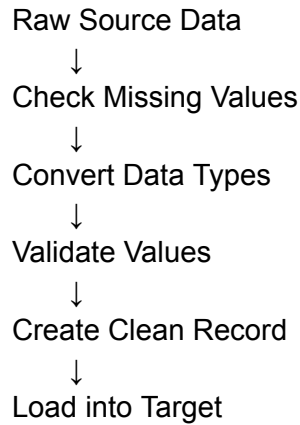
After cleaning:

```
{
  "customer_id": 101,
  "customer_name": "Anurag",
  "order_amount": 2500.5,
  "is_active": True,
  "discount": None,
  "last_order_date": None
}
```

This cleaned record is much more suitable for validation and loading into the target.

16. Real Pipeline Flow

A simplified source-cleaning process looks like:



The source may be:

- CSV
- JSON
- API
- Database
- Message queue
- Excel file
- User form
- External application

The exact source can change, but the cleaning principle remains similar.

17. Keep Raw and Cleaned Values Separate

Instead of immediately overwriting the source:

```
customer_id = int(customer_record["customer_id"])
```

you may keep:

```
raw_customer_id = customer_record["customer_id"]  
customer_id = int(raw_customer_id)
```

This can help with:

- Debugging
- Validation
- Auditing
- Investigating rejected records

This becomes especially useful when a conversion fails.

18. Common Mistakes to Avoid

1. Assuming a Number-Looking Value Is Numeric

```
customer_id = "101"
```

This is still a string.

Use `type()` when unsure.

2. Directly Using `bool()` on Source Text

Wrong assumption:

```
bool("False")
```

Result:

True

Use explicit comparison instead.

3. Treating `"NULL"` as `None`

```
"NULL"
```

is a string.

None

means no value.

4. Treating Zero as Missing

discount = 0

may be completely valid.

Always apply the business rule first.

5. Converting Without Validation

int("ABC")

will fail.

Source data cannot always be trusted to contain the expected format.

6. Destroying the Raw Value Too Early

Keeping raw and cleaned values separately can make troubleshooting much easier.

19. Quick Revision

- Source systems often send values as strings.
- Use `int()` for whole-number conversion.
- Use `float()` for decimal-number conversion.
- Use `str()` when text representation is required.
- Do not blindly use `bool()` on source strings.
- `"False"` is a non-empty string, so `bool("False")` returns `True`.
- `None` represents no value.

- "", "NULL" and "None" are strings.
 - Zero and False may be valid business values.
 - Standardise different missing-value formats.
 - Validate source values before conversion.
 - Preserve raw data when it helps debugging and auditing.
-

20. Student Practice Assignment

Clean the following source record:

```
order_record = {  
    "order_id": "5001",  
    "customer_id": "201",  
    "order_amount": "3500.75",  
    "discount": "NULL",  
    "is_priority": "Yes",  
    "retry_count": "2",  
    "error_message": ""  
}
```

Expected types:

```
order_id      → int  
customer_id   → int  
order_amount  → float  
discount      → float or None  
is_priority   → bool  
retry_count   → int  
error_message → str or None
```

Expected cleaned structure:

```
{  
    "order_id": 5001,  
    "customer_id": 201,  
    "order_amount": 3500.75,  
    "discount": None,  
    "is_priority": True,  
    "retry_count": 2,  
}
```

```
"error_message": None
}
```

After cleaning, print the type of every value.

21. Practice Challenge

Try the same process with:

```
order_record = {
    "order_id": "5002",
    "customer_id": "202",
    "order_amount": "INVALID",
    "discount": "250.50",
    "is_priority": "No",
    "retry_count": "0",
    "error_message": None
}
```

Identify:

- Which field causes the conversion failure?
- Which values can still be converted?
- Which value is a valid zero?

Do not handle the error yet. Observe what Python reports.

22. Interview-Relevant Questions

1. What is type conversion?

Type conversion is changing a value from one Python data type into another.

Example:

```
record_count = int("1500")
```

2. Why is type conversion important in Data Engineering?

External systems may send numbers, Boolean values and other fields as strings. They need to be converted before calculations, validations or loading into the expected target schema.

3. What is the difference between **None** and **"None"**?

None

represents the absence of a value.

"None"

is a normal string.

4. What does **bool("False")** return?

True

Because **"False"** is a non-empty string.

5. How can **"False"** be safely converted?

```
value = "False"
```

```
boolean_value = value.strip().lower() == "true"
```

Result:

False

6. Is zero considered a missing value?

No.

Zero may represent valid information such as:

- Zero retries
- Zero discounts
- Zero failed records

It should only be considered missing if the business rule explicitly defines it that way.

7. Why standardise missing values?

A source may represent missing values as:

""

"NULL"

"None"

"N/A"

None

Converting them to one consistent representation such as `None` makes downstream validation and processing easier.

8. Why can type conversion fail?

Conversion fails when the source value cannot be represented by the requested type.

Example:

```
int("ABC101")
```

cannot produce an integer.

9. Why keep raw and cleaned values separately?

It makes debugging, auditing, validation and rejected-record analysis easier.

23. Interview Practice: Predict the Output

```
value_1 = bool("")  
value_2 = bool("False")  
value_3 = bool(0)  
value_4 = bool(100)
```

```
print(value_1)  
print(value_2)  
print(value_3)  
print(value_4)
```

Expected output:

```
False  
True  
False  
True
```

Remember:

- Empty string → Falsy
- Non-empty string → Truthy
- Zero → Falsy
- Non-zero number → Truthy